

Respuestas Preguntas

1. ¿Qué diferencias has observado entre las 5 soluciones?

La ejecución secuencial y la ``async/await`` dan tiempos similares puesto que aunque por dentro trabajen distinto, ambas siguen esperando una acción detrás de otra, así que el tiempo total no cambia. Por tanto, el comportamiento observable desde fuera es idéntico.

La versión paralela es la que marca la diferencia. Aquí lo que se hace es lanzar varias cadenas de trabajo a la vez, haciendo uso de la asincronía y el paralelismo. Además, gracias al ``Task.WhenAll`` se puede esperar solo a la tarea más lenta.

2. ¿Qué acciones se pueden ejecutar a la vez y cuáles no? ¿Por qué?

Hay 4 cadenas principales que se pueden ejecutar simultáneamente: café, sartén, tostada y zumo.

Dentro de la cadena 2, huevos y bacon sí pueden ir en paralelo entre ellos una vez la sartén está caliente, pero ambos necesitan esperar a que la sartén termine primero.

Dentro de la cadena 3, la tostada primero requiere de tostar el pan y luego de este ser untado.

Esto es así ya que el enunciado lo estipula así.

3. ¿Qué ha pasado con cada solución cuando introduces el timeout?

En las versiones secuencial y `async/await`, el timeout de 500 ms se cumple mucho antes de que el trabajo termine (que tarda ~1500 ms), así que siempre se muestra el mensaje de "café frío". En la versión paralela, el trabajo (~515 ms) y el límite (500 ms) quedan tan cerca que el resultado cambia entre ejecuciones: a veces termina a tiempo, a veces no.

4. ¿El enfoque con mejor rendimiento es también el más seguro? ¿Por qué?

No necesariamente. La versión paralela con timeout lo demuestra bien: al llevar el tiempo de ejecución (~515 ms) muy cerca del límite (500 ms), cualquier variación pequeña puede decidir si el resultado es un éxito o un timeout. Ir al límite del rendimiento máximo elimina el margen de seguridad, por lo que un enfoque con menor rendimiento podría llegar a ser más predecible.

5. ¿Merece la pena complicarse con paralelismo o con mecanismos de control de tiempo? Justifica tu respuesta.

Con el paralelismo desde luego que sí, en este ejercicio se ve como se reduce el tiempo de carga en $1/3$. Es sin duda algo muy rentable, quizás no tanto en tiempos de carga muy bajos, pero en cuanto a un problema real con tiempos de carga más elevados no usar paralelismo sería un error de diseño y algo que podría costar mucho dinero mal invertido.

En cuanto a los mecanismos de control de tiempo diría que depende. Para este ejemplo estos mecanismos son muy útiles porque nos permiten ver con claridad los tiempos de carga de cada proceso. El problema vendría en un problema real con el `'Task.WaitAny'`, ya que esté solo controla el tiempo de espera, no cancela el trabajo de verdad.